1 Simply Typed Lambda Calculus

1.1 Syntax

Taking inspiration from [1] and [2] we define the types of the simply typed lambda calculus as:

$$T \coloneqq o \mid i \mid T \to T$$

That is, o is a type, i is a type and given two types A and B, the expression $A \to B$ is also a type. The type o represents the type of propositions, i represents the type of individuals and $A \to B$ represents the type of functions from A to B.

Some examples of types:

$$o \qquad o \to i \qquad o \to i \to o \qquad (o \to i) \to o$$

We consider $\to$ to be right associative, and thus $o \to i \to o$ is the same as $o \to (i \to o)$.

We define the terms of the simply typed lambda calculus (using the Bruijn levels) as:

$$t \coloneqq n \mid t\,t \mid \lambda_T\, t \mid \alpha \mid \forall_T \mid \exists_T$$

where n is a natural number, T is a type and α is one of the following symbols:

$$\alpha \coloneqq \bot \mid \top \mid \wedge \mid \vee \mid \supset$$

Some examples of terms:

$$\lambda_i\, 0 \qquad (\lambda_{i\to o}\, 2)\, 1 \qquad \lambda_o\, 3\ \vee \qquad \lambda_{o\to o}\, \lambda_i\, 0\, \top \qquad \forall_o\, \bot$$

The interpretation of a term depends on a fixed number m of free variables. All the levels which are less then m represent free variables, while the others represent bound variables. For example, if the free variables are 2 (that is, the levels 0 and 1), the terms $\lambda_i\, 0$ and $(\lambda_{i\to o}\, 2)\, 1$ can be interpreted respectively as $\lambda\, x\colon i, y$ and $(\lambda\, x\colon i \to o, x)\, y$. If the free variables are 0, then $\lambda_i\, 0$ corresponds to $\lambda\, x\colon i, x$, while the term $(\lambda_{i\to o}\, 2)\, 1$ makes no sense, because 1 and 2 should be two distinct bound variables.

1.2 Typing

Given a list of types Γ, a natural number n and a type T, we denote the n-th element of Γ by Γ_n and the list constructed by appending the element T to the end of Γ as Γ, T.

Definition 1. Given a list of types Γ, a term t and a type T, we define the

proposition $\Gamma \vdash t : T$ by the following inference rules:

$$\frac{}{\Gamma \vdash n : T} \quad \text{if } \Gamma_n = T$$

$$\frac{\Gamma \vdash s : A \to B \qquad \Gamma \vdash t : A}{\Gamma \vdash s\,t : B}$$

$$\frac{\Gamma, A \vdash t : B}{\Gamma \vdash \lambda_A\, t : A \to B}$$

$$\frac{}{\Gamma \vdash \bot : o} \qquad \frac{}{\Gamma \vdash \top : o}$$

$$\frac{}{\Gamma \vdash \wedge : o \to o \to o} \qquad \frac{}{\Gamma \vdash \vee : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \supset : o \to o \to o}$$

$$\frac{}{\Gamma \vdash \forall_T : (T \to o) \to o}$$

$$\frac{}{\Gamma \vdash \exists_T : (T \to o) \to o}$$

If the proposition $\Gamma \vdash t : T$ holds, we say that t *has type* T in the *context* Γ.

Some examples:

$$\frac{}{[i;\, o] \vdash 0 : i}$$

$$\cfrac{\cfrac{\cfrac{}{[i \to o;\, i] \vdash 0 : i \to o} \qquad \cfrac{}{[i \to o;\, i] \vdash 1 : i}}{[i \to o;\, i] \vdash 0\,1 : o}}{\cfrac{i \to o \vdash \lambda_i\, 0\,1 : i \to o}{\vdash \lambda_{i \to o}\, \lambda_i\, 0\,1 : (i \to o) \to i \to o}}$$

1.3 Substitution

The substitution of the terms $s_0, \ldots, s_n$ with context m into the term t, indicated as $t\,[s_0, \ldots, s_n]_m$, is defined recursively as follows:

$$\begin{aligned}
i\,[s_0, \ldots, s_n]_m &\coloneqq s_i \quad \text{if } i \in \mathbb{N} \text{ and } i \leq n \\
i\,[s_0, \ldots, s_n]_m &\coloneqq i \quad \text{if } i \in \mathbb{N} \text{ and } i > n \\
(t_1\, t_2)\,[s_0, \ldots, s_n]_m &\coloneqq (t_1\,[s_0, \ldots, s_n]_m)\,(t_2\,[s_0, \ldots, s_n]_m) \\
(\lambda_T\, t)\,[s_0, \ldots, s_n]_m &\coloneqq \lambda_T\,(t\,[s_0^+, \ldots, s_n^+, m]_{m+1}) \\
t\,[s_0, \ldots, s_n]_m &\coloneqq t \quad \text{in all other cases}
\end{aligned}$$

where s_k^+ is the term obtained from s_k by increasing all of its bound levels by one, with respect to a context with m free variables.

Some examples:

$$
\begin{aligned}
(1\,0)\,[\lambda_i\,0;\,1]_2 &= (1\,[\lambda_i\,0;\,1]_2)\,(0\,[\lambda_i\,0;\,1]_2) = 1\,(\lambda_i\,0)\\
(\lambda_o\,1\,0)\,[\lambda_i\,0;\,1]_2 &= \lambda_o\,((1\,0)\,[\lambda_i\,0;\,1;\,2]_3) = \lambda_o\,1\,(\lambda_i\,0)\\
(\lambda_o\,1\,0)\,[\lambda_i\,0]_2 &= \lambda_o\,((1\,0)\,[\lambda_i\,0;\,2]_3) = \lambda_o\,2\,(\lambda_i\,0)\\
(\lambda_o\,0)\,[\lambda_i\,0\,1]_2 &= \lambda_o\,(0\,[\lambda_i\,0\,1;\,2]_3) = \lambda_o\,\lambda_i\,0\,1\\
(\lambda_o\,0)\,[\lambda_i\,0\,1]_1 &= \lambda_o\,(0\,[\lambda_i\,0\,2;\,1]_2) = \lambda_o\,\lambda_i\,0\,2\\
(\lambda_o\,0)\,[\lambda_i\,0\,0]_0 &= \lambda_o\,(0\,[\lambda_i\,1\,1;\,0]_1) = \lambda_o\,\lambda_i\,1\,1
\end{aligned}
$$

In a substitution $t\,[s_0,\ldots,s_n]_m$, the resulting term and all the terms $s_0,\ldots,s_n$ are meant to be interpreted with m free variables, while the term t is meant to be interpreted with $n+1$ free variables.

Theorem 1. *Let $T_1,\ldots,T_m$ and $S_1,\ldots,S_n$ be types and let t and $s_1,\ldots,s_n$ be terms such that $[T_1;\,\ldots;\,T_m]\vdash s_i : S_i$ for every $i=1,\ldots,n$. If $[S_1;\,\ldots;\,S_n]\vdash t : T$ then $[T_1;\,\ldots;\,T_m]\vdash t\,[s_1;\,\ldots;\,s_n]_m : T$.*

Theorem 2. *Let $T_1,\ldots,T_m$ and $S_1,\ldots,S_n$ be types and let t and $s_1,\ldots,s_n$ be terms such that $[T_1;\,\ldots;\,T_m]\vdash s_i : S_i$ for every $i=1,\ldots,n$. If $m\le n$ and $[T_1;\,\ldots;\,T_m]\vdash t\,[s_1;\,\ldots;\,s_n]_m : T$ then $[S_1;\,\ldots;\,S_n]\vdash t : T$.*

For example, $[o]\vdash 0 : o$ and $[i]\vdash \top : o$ and $[i]\vdash 0\,[\top]_1 : o$, that is, $[i]\vdash \top : o$.

1.4 Reduction

We define the β-reduction of a term recursively as follows:

$$
\begin{aligned}
\mathrm{red}^m_\beta(\lambda_T\,t)\,s &:= \left(\mathrm{red}^{m+1}_\beta t\right)[0;\,\ldots;\,m-1;\,\mathrm{red}^m_\beta s]_m\\
\mathrm{red}^m_\beta\,s\,t &:= \left(\mathrm{red}^m_\beta s\right)\left(\mathrm{red}^m_\beta t\right)\\
\mathrm{red}^m_\beta(\lambda_T\,t) &:= \lambda_T\left(\mathrm{red}^{m+1}_\beta t\right)\\
\mathrm{red}^m_\beta t &:= t
\end{aligned}
$$

We define the η-reduction of a term recursively as follows:

$$
\begin{aligned}
\mathrm{red}^m_\eta\,\lambda_T\,t\,m &:= t^-\\
\mathrm{red}^m_\eta t &:= t
\end{aligned}
$$

where t^- is the term obtained from t by decreasing all of its bound levels by one, with respect to a context with m free variables.

2 G Logic

Given a list of types σ and a term t, the expression $\sigma\triangleright t$ is called a *judgement*. Let Σ be a list of types, Γ a list of judgements and B a judgement. The expression $\Sigma;\Gamma\longrightarrow B$ is called a *sequent*. We denote the list constructed by appending a head element B to a tail Γ as B,Γ. The concatenation of two lists Σ and σ is denoted by $\Sigma\,\sigma$.

We define a calculus with the following inference rules:

$$\frac{\Sigma\,\sigma \vdash t : o}{\Sigma; [\sigma \rhd t] \longrightarrow \sigma \rhd t}\ \text{init} \qquad \frac{\Sigma; \Delta \longrightarrow B \qquad \Sigma; B, \Gamma \longrightarrow C}{\Sigma; \Delta\,\Gamma \longrightarrow C}\ \text{cut}$$

$$\frac{\Sigma; B, B, \Gamma \longrightarrow C}{\Sigma; B, \Gamma \longrightarrow C}\ cL \qquad \frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \Gamma \longrightarrow C}{\Sigma; \sigma \rhd t, \Gamma \longrightarrow C}\ wL$$

$$\frac{\Sigma\,\sigma \vdash t : o}{\Sigma; [\rho \rhd \bot] \longrightarrow \sigma \rhd t}\ \bot L \qquad \frac{}{\Sigma; [\,] \longrightarrow \sigma \rhd \top}\ \top R$$

$$\frac{\Sigma\,\sigma \vdash s : o \qquad \Sigma; \sigma \rhd t, \Gamma \longrightarrow B}{\Sigma; \sigma \rhd t \wedge s, \Gamma \longrightarrow B}\ \wedge L \qquad \frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \sigma \rhd s, \Gamma \longrightarrow B}{\Sigma; \sigma \rhd t \wedge s, \Gamma \longrightarrow B}\ \wedge L$$

$$\frac{\Sigma; \Gamma \longrightarrow \sigma \rhd s \qquad \Sigma; \Gamma \longrightarrow \sigma \rhd t}{\Sigma; \Gamma \longrightarrow \sigma \rhd s \wedge t}\ \wedge R$$

$$\frac{\Sigma; \sigma \rhd s, \Gamma \longrightarrow B \qquad \Sigma; \sigma \rhd t, \Gamma \longrightarrow B}{\Sigma; \sigma \rhd s \vee t, \Gamma \longrightarrow B}\ \vee L$$

$$\frac{\Sigma\,\sigma \vdash t : o \qquad \Sigma; \Gamma \longrightarrow \sigma \rhd s}{\Sigma; \Gamma \longrightarrow \sigma \rhd s \vee t}\ \vee R \qquad \frac{\Sigma\,\sigma \vdash s : o \qquad \Sigma; \Gamma \longrightarrow \sigma \rhd t}{\Sigma; \Gamma \longrightarrow \sigma \rhd s \vee t}\ \vee R$$

$$\frac{\Sigma; \Gamma \longrightarrow \sigma \rhd s \qquad \Sigma; \sigma \rhd t, \Gamma \longrightarrow B}{\Sigma; \sigma \rhd s \supset t, \Gamma \longrightarrow B}\ \supset L \qquad \frac{\Sigma; \sigma \rhd s, \Gamma \longrightarrow \sigma \rhd t}{\Sigma; \Gamma \longrightarrow \sigma \rhd s \supset t}\ \supset R$$

$$\frac{\Sigma\,\sigma \vdash t : T \qquad \Sigma; \sigma \rhd s\,[0;\,1;\,\ldots;\,|\Sigma\,\sigma| - 1;\,t]_{|\Sigma\,\sigma|}, \Gamma \longrightarrow B}{\Sigma; (\sigma \rhd \forall_T\, s), \Gamma \longrightarrow B}\ \forall L$$

$$\frac{\Sigma, (T_{|\Sigma|+1} \to \ldots \to T_n \to S); \Gamma \longrightarrow [T_{|\Sigma|+1};\,\ldots;\,T_n] \rhd s'}{\Sigma; \Gamma \longrightarrow [T_{|\Sigma|+1};\,\ldots;\,T_n] \rhd \forall_S\, s}\ \forall R$$

$$\frac{\Sigma, (T_{|\Sigma|+1} \to \ldots \to T_n \to S); [T_{|\Sigma|+1};\,\ldots;\,T_n] \rhd s', \Gamma \longrightarrow B}{\Sigma; ([T_{|\Sigma|+1};\,\ldots;\,T_n] \rhd \exists_S\, s), \Gamma \longrightarrow B}\ \exists L$$

$$\frac{\Sigma\,\sigma \vdash t : T \qquad \Sigma; \Gamma \longrightarrow \sigma \rhd s\,[0;\,1;\,\ldots;\,|\Sigma\,\sigma| - 1;\,t]_{|\Sigma\,\sigma|}}{\Sigma; \Gamma \longrightarrow \sigma \rhd \exists_T\, s}\ \exists R$$

where

$$s' = s\,[0;\,\ldots;\,|\Sigma| - 1;\,|\Sigma| + 1;\,\ldots;\,n;\,|\Sigma|\,(|\Sigma| + 1)\,\ldots\,n]_{n+1}$$

We want for every term that occurs inside a sequent to have type o. We check that the all the rules enforce this, focusing on the non trivial ones only.

$$\cfrac{\cfrac{\Sigma\,\sigma \vdash t : T \qquad \Sigma\,\sigma \vdash s\,[0;\,1;\,\ldots;\,|\Sigma\,\sigma| - 1;\,t]_{|\Sigma\,\sigma|} : o}{\Sigma\,\sigma, T \vdash s : o}}{\Sigma\,\sigma \vdash \forall_T\, s : o}$$

References

[1] Alonzo Church. "A Formulation of the Simple Theory of Types". In: *The Journal of Symbolic Logic* (1940). URL: https://www.jstor.org/stable/2266170.

[2] Benjamin C. Pierce et al. *Software Foundations, Volume 1: Programming Language Foundations*. 2026. URL: https://softwarefoundations.cis.upenn.edu/plf-current.